NVIDIA-Certified Associate

Generative AI LLMs

Master Cheat Sheet

Organized quick-review glossary for exam recall, scenario mapping, and final revision.

Prepared by: Yasir Alharbi

Cybersecurity and AI enthusiast

LinkedIn: www.linkedin.com/in/Yasir-T-Alharbi

How to use: scan domains first, then use decision tree and scenario keywords for exam questions.

Main Sections

Consolidated from the provided PDFs and domain screenshots. Definitions are intentionally short for fast exam review.

Exam Domains

Core ML & AI Fundamentals

Weight: 30%

Concepts

- **Artificial Intelligence (AI):** Systems that perform tasks requiring human intelligence.
- **Machine Learning (ML):** Algorithms learn patterns from data.
- **Deep Learning (DL):** Multi-layer neural networks learn representations automatically.
- **Generative AI:** AI that creates text, images, code, audio, or data.
- **Foundation Model:** Large pretrained model adaptable to many tasks.
- **Large Language Model (LLM):** Transformer model trained on large text corpora.
- **Supervised Learning:** Learns from labeled input-output examples.
- **Unsupervised Learning:** Finds patterns in unlabeled data.
- **Semi-supervised Learning:** Uses small labeled data plus large unlabeled data.
- **Self-supervised Learning:** Creates labels from raw data itself.
- **Reinforcement Learning:** Learns actions using rewards.
- **Transfer Learning:** Reuses pretrained knowledge for a new task.
- **Feature Extraction:** Freeze base model; train only final layers.
- **Fine-tuning:** Continue training a pretrained model on task data.
- **Feature Engineering:** Create useful model inputs from raw data.
- **Representation Learning:** Model learns useful internal features.
- **Embedding:** Dense vector representing meaning or features.
- **Text Embedding:** Vector representation of text semantics.
- **Word2Vec:** Learns word embeddings from text context.
- **CBOW:** Predicts target word from surrounding context.
- **Skip-gram:** Predicts context words from a target word.
- **Overfitting:** Performs well on training data, poorly on unseen data.
- **Underfitting:** Model too simple to learn patterns.
- **Regularization:** Reduces overfitting.
- **Dropout:** Randomly disables neurons during training.
- **L1 Regularization:** Encourages sparse weights.
- **L2 Regularization:** Penalizes large weights.
- **Loss Function:** Measures prediction error.
- **Gradient Descent:** Updates weights to reduce loss.
- **Learning Rate:** Step size for weight updates.

Backpropagation: Computes gradients through network layers.
Epoch: One full pass through training data.
Batch Size: Samples processed per training step.
Hyperparameter: Config set before training.
Parameter: Learned model weight.
Inference: Running a trained model to produce output.
Classification: Predicts categories.
Regression: Predicts continuous values.
Clustering: Groups similar unlabeled data.
Anomaly Detection: Finds unusual data points.
Association Rule Mining: Finds item co-occurrence rules.
Time Series Forecasting: Predicts future ordered values.
Data Augmentation: Creates varied training examples.
Normalization: Scales values for stable training.
Standardization: Centers/scales features by mean and variance.
Padding: Adds tokens/pixels to standardize size.
Masking: Prevents attention to irrelevant positions.

Architectures

Dense Network: Fully connected neural network.
CNN: Learns spatial features from images.
Convolution: Sliding filter extracts local patterns.
Kernel/Filter: Small matrix detecting image features.
Stride: Step size of convolution movement.
Pooling: Reduces spatial size.
Max Pooling: Keeps strongest local activation.
Average Pooling: Averages local activations.
RNN: Processes sequential data step by step.
LSTM: RNN variant for long dependencies.
GRU: Simpler gated RNN variant.
Transformer: Attention-based architecture for sequences.
Encoder-only Model: Reads input for understanding tasks.
Decoder-only Model: Generates text autoregressively.
Encoder-decoder Model: Converts input sequence to output sequence.
BERT: Bidirectional encoder model.
GPT: Decoder-only autoregressive model.
T5: Encoder-decoder text-to-text model.

BART: Encoder-decoder generation model.
Vision Transformer (ViT): Treats image patches like tokens.
Autoencoder: Compresses and reconstructs data.
VAE: Generative autoencoder with latent distribution.
GAN: Generator and discriminator compete.
Diffusion Model: Learns to denoise step by step.
CLIP: Aligns image and text embeddings.
GNN: Learns from graph nodes and edges.
Multimodal LLM: Processes multiple data types together.

Transformer Concepts

Self-attention: Tokens attend to other tokens in same sequence.
Cross-attention: Decoder attends to encoder outputs.
Multi-head Attention: Multiple attention views in parallel.
Query (Q): Vector used to search attention targets.
Key (K): Vector used for attention matching.
Value (V): Vector containing retrieved information.
Scaled Dot-product Attention: Attention using QK scores scaled by sqrt(dk).
sqrt(dk) Scaling: Prevents large dot products destabilizing softmax.
Positional Encoding: Adds token order information.
Sinusoidal Encoding: Original fixed transformer position signal.
RoPE: Rotary position embedding for long contexts.
Causal Mask: Prevents decoder from seeing future tokens.
Padding Mask: Ignores padded tokens.
Feed-forward Network (FFN): Per-token nonlinear transformation.
Residual Connection: Adds input back to layer output.
LayerNorm: Normalizes activations inside transformer layers.
RMSNorm: Efficient LayerNorm alternative.
Context Window: Maximum tokens a model can process.
KV Cache: Stores attention keys/values for faster decoding.
Grouped-query Attention: Shares KV heads to reduce inference memory.
Sliding Window Attention: Attends to local token windows.
FlashAttention: Faster memory-efficient attention.
Mixture of Experts (MoE): Activates only selected expert submodels.

Applications

RAG Systems: Retrieve knowledge before generation.

Chatbots: Conversational LLM applications.
Summarizers: Shorten text while preserving meaning.
Translation: Convert text between languages.
Sentiment Analysis: Classify emotional tone.
NER: Detect named entities in text.
Token Classification: Label each token.
Text Classification: Label whole text.
Question Answering: Answer from context or generation.
Extractive QA: Selects answer span from passage.
Generative QA: Produces free-form answer.
Image Classification: Predicts image class.
Object Detection: Finds objects in images.
Speech Recognition: Converts audio to text.
Fraud Detection: Flags suspicious behavior.
Customer Churn Prediction: Predicts customer loss.
Recommendation Systems: Suggest items/users/content.
Drug Discovery: Uses models for molecules/proteins.

Frameworks, Libraries, Platforms, and Tools Map

PyTorch: Flexible deep learning framework.
TensorFlow: Production deep learning framework.
Keras: High-level TensorFlow model API.
JAX: High-performance autodiff/array framework.
scikit-learn: Classical ML and metrics toolkit.
Hugging Face Transformers: Pretrained transformer model library.
Hugging Face Datasets: Dataset loading/preprocessing library.
Hugging Face PEFT: Parameter-efficient fine-tuning tools.
Hugging Face TRL: RLHF/DPO training utilities.
spaCy: Production NLP pipeline library.
NLTK: Traditional NLP toolkit.
SentenceTransformers: Embedding models for semantic search.
LangChain: Framework for LLM apps and chains.
LangGraph: Stateful graph workflows for agents.
LlamaIndex: Data connectors and RAG framework.
NeMo Toolkit: NVIDIA LLM customization framework.
NeMo Curator: Dataset curation and filtering pipeline.
NeMo Guardrails: Safety/rail framework for LLM apps.

NVIDIA NIM: Optimized inference microservice containers.
TensorRT-LLM: Optimized LLM inference library.
Triton Inference Server: Production model serving platform.
RAPIDS: GPU data science Python ecosystem.
cuDF: GPU DataFrame library.
cuML: GPU machine learning algorithms.
cuGraph: GPU graph analytics.
Dask-cuDF: Distributed GPU DataFrames.
NVIDIA DALI: GPU data loading/preprocessing.
CuPy: NumPy-like arrays on GPU.
Numba: JIT compiler for Python/GPU kernels.
FAISS: Local vector similarity search.
Milvus: Open-source vector database.
Pinecone: Managed vector database.
Weaviate: Semantic vector database.
Chroma: Lightweight local vector database.
ONNX: Open model exchange format.
ONNX Runtime: Runs ONNX models.
Docker: Container runtime.
Kubernetes: Container orchestration system.
Helm: Kubernetes package manager.
FastAPI: Modern Python REST API framework.
Flask: Lightweight Python web framework.
Streamlit: Quick Python data apps.
Gradio: Quick ML demos/interfaces.
MLflow: Experiment and model tracking.
Weights & Biases: Experiment tracking dashboard.
DVC: Data/model version control.
Ray: Distributed Python computing.
Ray Serve: Scalable model serving.
BentoML: Package and serve ML models.
Airflow: Workflow orchestration.
Prefect: Python workflow orchestration.
NumPy: Python numeric computing package.
Pandas: CPU DataFrame analysis library.
Polars: Fast DataFrame processing library.
SciPy: Scientific computing and optimization.

Matplotlib: Basic Python plotting.
Seaborn: Statistical visualization library.
Plotly: Interactive visualization library.
Jupyter Notebook: Interactive notebooks.
pytest: Python testing framework.
Great Expectations: Data quality validation.
Evidently: ML drift and monitoring reports.

Data Analysis Techniques

Weight: 14%

Techniques

Data Mining: Extracts patterns from large data.
Data Visualization: Shows insights with charts.
Statistical Metrics: Quantify model/data behavior.
Trend Analysis: Finds patterns over time.
Performance Comparison: Compares models using metrics.
Exploratory Data Analysis: Understands distributions and relationships.
Descriptive Statistics: Summarizes data.
Correlation Analysis: Measures relationship between variables.
Distribution Analysis: Studies data shape/spread.
Outlier Detection: Finds abnormal values.
Dimensionality Reduction: Compresses features while preserving structure.
PCA: Linear dimensionality reduction.
Nearest Neighbor Search: Finds closest vectors/items.
Cosine Similarity: Compares vector orientation.
Euclidean Distance: Measures straight-line distance.
Clustering Analysis: Groups similar records.
K-means: Clusters around centroids.
Hierarchical Clustering: Builds nested cluster tree.
Spectral Clustering: Uses graph structure for clustering.
Graph Analytics: Analyzes nodes and edges.
Knowledge Graph: Structured entity-relation graph.

Processes

Data Collection: Gather raw data.
Data Cleaning: Fix missing, duplicate, inconsistent data.
Deduplication: Removes repeated records.

Filtering: Removes low-quality or irrelevant data.

Data Transformation: Convert data into model-ready form.

Feature Scaling: Normalize numeric ranges.

Categorical Encoding: Convert categories to numbers.

Tokenization: Break text into tokens.

Chunking: Split documents for retrieval/context limits.

Data Labeling: Assign ground-truth labels.

Data Versioning: Track dataset changes.

Dataset Provenance: Track data origin and usage rights.

Train/Validation/Test Split: Separate training, tuning, final testing.

Insight Generation: Turn analysis into action.

Tools

RAPIDS: GPU-accelerated data science suite.

cuDF: GPU DataFrame library; pandas-like.

cuML: GPU ML algorithms.

cuGraph: GPU graph analytics.

Dask-cuDF: Distributed multi-GPU DataFrames.

NVTabular: GPU feature engineering for tabular data.

CUDA-X: NVIDIA accelerated software libraries.

NVIDIA DALI: GPU data loading/preprocessing.

TAO Toolkit: Simplifies training and augmentation.

Pandas: CPU data manipulation.

Polars: Fast DataFrame library.

Dask: Distributed Python data processing.

Experimentation & Evaluation

Weight: 22%

Methods

A/B Testing: Compare versions on real traffic.

Benchmarking: Compare against standard baselines.

Cross-validation: Evaluate across multiple data splits.

K-fold Validation: Split data into k train/test folds.

Hold-out Testing: Use separate unseen test set.

Temporal Validation: Test using future time data.

Statistical Testing: Checks significance of results.

Human Evaluation: Humans judge quality.

LLM-as-a-Judge: LLM scores outputs.

Error Analysis: Systematically inspect failures.

Ablation Study: Remove components to measure impact.

Stress Testing: Test under heavy/extreme conditions.

Adversarial Testing: Test with hostile edge cases.

Red Teaming: Attack model to find risks.

Canary Deployment: Release to small user subset.

Metrics

Accuracy: Fraction correct.

Precision: Correct positives among predicted positives.

Recall: Correct positives among actual positives.

F1 Score: Harmonic mean of precision and recall.

Confusion Matrix: Counts prediction categories.

Specificity: Correct negatives among actual negatives.

AUC-ROC: Ranking quality across thresholds.

MAE: Average absolute regression error.

MSE: Average squared regression error.

RMSE: Square root of MSE.

R-squared: Regression variance explained.

Cross-entropy Loss: Classification probability loss.

BLEU: N-gram overlap for translation.

ROUGE: Overlap metric for summaries.

METEOR: Translation quality metric.

Perplexity: Language model uncertainty.

Groundedness: Output supported by sources.

Faithfulness: Response matches retrieved evidence.

Relevance: Answer addresses the query.

Coherence: Output is logically organized.

Toxicity Score: Harmful language measurement.

Latency: Time per request.

Throughput: Requests/tokens per time.

Error Rate: Failed requests percentage.

GPU Utilization: How much GPU is used.

RLHF

RLHF: Aligns LLM using human feedback.

Human Feedback: Human preference labels.
Preference Pair: Two outputs ranked by humans.
Reward Model: Predicts human preference score.
PPO: RL algorithm used in RLHF.
DPO: Directly optimizes preference data.
Reward Hacking: Model exploits reward incorrectly.
Safety Alignment: Makes outputs follow human values.
Constitutional AI: Uses written principles for alignment.
SFT: Supervised fine-tuning on instruction-response pairs.

Software Development

Weight: 24%

Development

Model Deployment: Serve models in production.
API Endpoint: Network interface for model calls.
REST API: HTTP-based service interface.
gRPC: High-performance service protocol.
SDK: Developer tools for using a platform.
Containerization: Package app and dependencies.
Docker: Container runtime.
Kubernetes: Container orchestration system.
Helm Chart: Kubernetes deployment package.
Version Control: Track code/model changes.
Git: Distributed version control tool.
CI/CD: Automated build, test, deployment.
Unit Testing: Tests small code units.
Integration Testing: Tests connected components.
Model Validation Pipeline: Automated model quality checks.
Configuration File: Stores settings/hyperparameters.
YAML/JSON Config: Common configuration formats.
Logging: Records events for debugging/audit.
Asynchronous Programming: Handles many I/O tasks concurrently.
Scalability: Handles growing workload.
Horizontal Scaling: Add more instances.
Vertical Scaling: Add more resources per instance.
Caching: Reuse frequent responses/results.

Frameworks and App Tools

LangChain: Framework for LLM apps.

LangGraph: Orchestrates complex LLM workflows.

ChatNVIDIA: LangChain connector for NVIDIA chat models.

NVIDIAEmbeddings: LangChain connector for NVIDIA embeddings.

Vector Database: Stores/searches embeddings.

FAISS: Vector similarity search library.

Milvus: Vector database.

Pinecone: Managed vector database.

Weaviate: Vector database with semantic search.

ONNX: Open model exchange format.

OpenAI-compatible API: Standard chat/completion API shape.

Monitoring

Performance Monitoring: Tracks latency/throughput/errors.

Model Monitoring: Tracks quality/drift.

Infrastructure Monitoring: Tracks CPU/GPU/memory.

Drift Detection: Finds distribution changes.

Quality Degradation Alert: Warns when model worsens.

Automated Alerts: Real-time issue notifications.

Audit Trail: Record of actions and outputs.

User Feedback Loop: Uses real usage feedback.

Trustworthy AI Principles

Weight: 10%

Ethics

Fairness: Equitable treatment across groups.

Transparency: Decisions are understandable.

Accountability: Clear responsibility for outcomes.

Privacy: Protect sensitive data.

Consent: Respect user data permission.

Beneficence: System should benefit people.

Robustness: Performs reliably under variation.

Explainability: Explains model decisions.

Compliance: Follows laws and regulations.

Model Card: Documents model purpose, limits, risks.

Data Sheet: Documents dataset details.

Bias

AI Bias: Unfair systematic model behavior.

Data Bias: Bias from training data.

Sampling Bias: Unrepresentative data collection.

Label Bias: Biased annotation/ground truth.

Algorithmic Bias: Bias introduced by model/design.

Evaluation Bias: Narrow tests hide failures.

Bias Detection: Measure unfair group differences.

Bias Mitigation: Reduce unfair outcomes.

Fairness Constraints: Enforce fairness during optimization.

Adversarial Debiasing: Train model to reduce bias signals.

Diverse Test Sets: Evaluate across groups/scenarios.

Safety and Guardrails

NeMo Guardrails: Controls safe LLM behavior.

Colang: Language for Guardrails dialogue flows.

Input Rail: Filters user input before LLM.

Output Rail: Filters response after LLM.

Topical Rail: Keeps conversation on allowed topics.

Fact-checking Rail: Verifies claims against sources.

Jailbreak: Prompt that bypasses safety rules.

Prompt Injection: Malicious instruction inside input/context.

Toxicity Detection: Finds harmful language.

PII Protection: Protects personal information.

Differential Privacy: Adds privacy-preserving noise.

Key NVIDIA Technologies

Platforms

NVIDIA NIM: GPU-optimized inference microservice containers.

NVIDIA NeMo: Framework for LLM training/fine-tuning/customization.

TensorRT-LLM: Optimizes LLM inference on NVIDIA GPUs.

Triton Inference Server: Serves models with batching/versioning.

RAPIDS: GPU-accelerated data science libraries.

cuDF: GPU DataFrame library.

cuML: GPU ML algorithms.

cuGraph: GPU graph analytics.

CUDA: NVIDIA GPU programming platform.

cuDNN: GPU primitives for deep neural networks.
NCCL: Multi-GPU communication library.
NGC: NVIDIA catalog for models, containers, SDKs.
BioNeMo: Foundation models for biology/life sciences.
Megatron-LM: Large-scale transformer training framework.
NeMo Curator: Data curation/preprocessing pipeline.
NeMo Guardrails: Safety framework for LLM apps.
NVIDIA AI Endpoints: Hosted NVIDIA model APIs.
DeepStream SDK: Real-time video AI streaming.
Omniverse: 3D simulation/visualization platform.
Omniverse Nucleus: Collaboration/storage for Omniverse assets.

NIM Details

NIM Container: Prebuilt optimized model server.
NGC Distribution: NIM images come from NGC.
OpenAI API Compatibility: Existing clients can switch easily.
REST/gRPC Endpoint: NIM exposes production APIs.
TensorRT-LLM inside NIM: Provides optimized inference.
Triton Backend: Many NIMs use Triton internally.
Hardware Portability: Same container across cloud/on-prem/edge.

NeMo Details

SFT in NeMo: Trains on prompt-response pairs.
RLHF in NeMo: Reward model plus PPO alignment.
LoRA in NeMo: Efficient adapter fine-tuning.
P-Tuning: Learns soft prompt vectors.
Prompt Tuning: Trains prompt embeddings.
Distributed Training: Scales across GPUs/nodes.
Custom Tokenizer: Domain-specific token splitting.
Megatron Integration: Enables tensor/pipeline parallelism.

TensorRT-LLM Details

In-flight Batching: Adds requests into active batches.
Continuous Batching: Keeps GPU busy during serving.
Paged KV Cache: Pages attention cache memory.
Custom CUDA Kernels: Faster low-level GPU operations.
Quantization: Lower precision for speed/memory.
INT8: 8-bit inference precision.

INT4: 4-bit compression precision.

FP8: Low precision floating-point.

AWQ: Activation-aware weight quantization.

GPTQ: Post-training quantization method.

Tensor Parallelism: Splits tensors across GPUs.

Triton Details

Dynamic Batching: Groups requests automatically.

Model Ensemble: Chains models into one pipeline.

Model Versioning: Hosts multiple versions.

Concurrent Execution: Runs multiple models together.

Multi-framework Serving: Supports PyTorch, TF, ONNX, TensorRT.

Model Repository: Directory of deployable models/configs.

A/B Testing: Compare model versions in production.

NGC Details

Model Catalog: Find pretrained models.

NIM Catalog: Get production containers.

Helm Charts: Deploy on Kubernetes.

Version Pinning: Reproducible container/model versions.

Model Signing: Verifies container integrity.

Security Scanning: Checks container vulnerabilities.

Prompt Engineering Mastery

Techniques

Prompt Engineering: Craft prompts for desired outputs.

Zero-shot Prompting: No examples provided.

Few-shot Prompting: Includes examples in prompt.

One-shot Prompting: Includes one example.

Chain-of-Thought: Encourages step-by-step reasoning.

Self-consistency: Sample multiple reason paths; choose consensus.

Tree of Thoughts: Explore multiple reasoning branches.

Role Prompting: Assign model a role/persona.

System Prompt: Sets top-level behavior/instructions.

Delimiters: Separate instructions from data.

Structured Output Prompting: Request JSON/tables/schema.

Output Parsing: Convert model output to usable format.

Prompt Chaining: Use outputs as later inputs.
Function Calling: Let LLM call external tools.
Multi-turn Conversation: Maintain dialogue context.
Domain Adaptation Prompting: Tailor prompt to domain.
Safety Prompting: Reduce harmful/off-policy responses.

Generation Controls

Temperature: Controls randomness.
Low Temperature: More deterministic output.
High Temperature: More creative output.
Top-k Sampling: Sample from k most likely tokens.
Top-p Sampling: Sample from probability nucleus.
Max Tokens: Limits output length.
Frequency Penalty: Discourages repeated tokens.
Repetition Penalty: Reduces repeated phrases.
Context Management: Use context window efficiently.
Token Efficiency: Minimize tokens while preserving task.
Iterative Refinement: Improve prompt based on output.

Model Evaluation & Testing

Evaluation

Automated Metrics: Quantitative model scores.
Human Evaluation: Expert/user quality judgment.
Benchmark Dataset: Standard comparison dataset.
GLUE: NLP understanding benchmark.
SuperGLUE: Harder NLP benchmark.
HELM: Holistic LLM evaluation benchmark.
MMLU: Multi-domain knowledge benchmark.
Domain-specific Test: Custom test for target use case.
Adversarial Test: Robustness against attacks/edge cases.
Regression Test: Ensures behavior does not break.
Golden Dataset: Trusted reference test set.

Validation

Cross-validation: Reliable performance estimate.
Hold-out Test: Final unbiased test set.
Temporal Validation: Future data validation.

Distribution Shift: Train/test data differ.

Stress Test: Performance under heavy load.

Calibration: Confidence matches correctness.

Generalization: Works on unseen examples.

Monitoring

Drift Detection: Input/output distribution changes.

Performance Tracking: Continuous metric monitoring.

Error Analysis: Investigate failure patterns.

User Feedback: Real-world quality signal.

Automated Alert: Notifies degradation.

Production Evaluation: Measures live system behavior.

Repeated Exam Patterns

Tool-to-Phase Mapping

Data Prep: RAPIDS, cuDF, DALI, NeMo Curator.

Training: NeMo, Megatron-LM, PyTorch, TensorFlow.

Fine-tuning: NeMo, SFT, LoRA, P-Tuning, Prompt Tuning.

Alignment: RLHF, reward model, PPO, DPO.

Optimization: TensorRT-LLM, quantization, batching, KV cache.

Serving: NIM, Triton, REST/gRPC.

App Development: LangChain, LangGraph, NVIDIA AI Endpoints.

Retrieval: Embeddings, vector DB, FAISS, Milvus.

Safety: NeMo Guardrails, Colang, rails.

Catalog: NGC.

Monitoring: Logs, drift, latency, throughput, alerts.

RAG Pipeline

User Query: User asks a question.

Query Embedding: Convert query to vector.

Vector Database: Stores searchable document vectors.

ANN Search: Fast approximate nearest neighbor retrieval.

Retriever: Finds relevant chunks.

Re-ranker: Reorders retrieved chunks by relevance.

Context Injection: Add chunks into prompt.

LLM Generation: Produce grounded answer.

Citation/Source Check: Verify answer support.

RAG Benefit: Reduces hallucination and adds fresh knowledge.
RAG Risk: Bad retrieval causes bad answers.

Fine-tuning Methods

Full Fine-tuning: Updates all model weights.
PEFT: Updates small trainable components.
LoRA: Low-rank trainable adapters.
QLoRA: Quantized base model plus LoRA.
Adapter Tuning: Small modules inserted in layers.
Prompt Tuning: Trains soft prompt tokens.
P-Tuning: Learns continuous prompt vectors.
Instruction Tuning: Teaches instruction following.
Domain Adaptation: Adapts model to specialized domain.
Catastrophic Forgetting: New training erases old skills.
Gradient Checkpointing: Saves memory by recomputing activations.
Mixed Precision Training: Uses FP16/BF16 for speed/memory.

Optimization and Deployment

Quantization: Reduces precision for efficient inference.
Pruning: Removes less important weights.
Distillation: Smaller model learns from larger model.
Batching: Process requests together.
Dynamic Batching: Triton groups concurrent requests.
Continuous/In-flight Batching: Adds requests mid-generation.
Speculative Decoding: Smaller draft model speeds generation.
KV Cache: Speeds autoregressive decoding.
Caching: Stores common responses/results.
Autoscaling: Adjusts replicas to demand.
Cold Start: Delay when starting service/model.
Edge Deployment: Run model near device/user.
Model Registry: Tracks model versions/artifacts.
Canary Release: Small rollout before full release.
Container Orchestration: Manage containers at scale.

Trust and Risk

Hallucination: Confident but false output.
Grounding: Connect output to evidence.
Jailbreak: Bypass safety guardrails.

Prompt Injection: Malicious prompt hidden in input.
Data Leakage: Sensitive data exposed.
Copyright Risk: Training data may violate rights.
Privacy Risk: User data mishandled.
Bias Risk: Unequal performance across groups.
Toxic Output: Harmful or offensive response.
Robustness Risk: Fails under unusual inputs.
Accountability Gap: Unclear responsibility for harm.

Data Types and Preprocessing

Image Data: Resize, normalize, augment.
Text Data: Tokenize, pad, embed.
Audio Data: Spectrograms, normalization, augmentation.
Tabular Data: Scale numbers, encode categories.
Time-series Data: Normalize, window, forecast.
Image Rotation: Augments image angle.
Image Flipping: Mirrors images.
Random Crop: Varies image view.
Color Jitter: Changes brightness/contrast/color.
Noise Injection: Adds random noise for robustness.
Synonym Replacement: Text augmentation by synonyms.
Back Translation: Translate out and back to paraphrase.
Random Insertion/Deletion: Adds/removes words.
Time Shift: Audio time augmentation.
Pitch Scaling: Audio pitch augmentation.

Common Confusions

NIM vs Triton: NIM is packaged model microservice; Triton is serving server.
NIM vs NGC: NIM runs models; NGC stores/distributes containers/models.
NeMo vs NIM: NeMo trains/customizes; NIM deploys/serves.
TensorRT-LLM vs Triton: TensorRT-LLM optimizes LLMs; Triton serves models.
Guardrails vs RLHF: Guardrails control runtime behavior; RLHF changes model alignment.
RAPIDS vs Pandas: RAPIDS/cuDF runs DataFrames on GPU; pandas runs on CPU.
RAG vs Fine-tuning: RAG retrieves knowledge; fine-tuning changes weights.
LoRA vs Full Fine-tuning: LoRA updates adapters; full updates all weights.
BLEU vs ROUGE: BLEU often translation; ROUGE often summarization.
Precision vs Recall: Precision avoids false positives; recall avoids false negatives.

Latency vs Throughput: Latency is delay; throughput is volume.
Encoder vs Decoder: Encoder understands; decoder generates.
Self-attention vs Cross-attention: Same sequence vs encoder-decoder attention.

AI Subject Terms Glossary

Core AI Terms

Agent: System that acts toward a goal.
AI Pipeline: Steps from data to deployed model.
Algorithm: Procedure for solving a task.
Baseline: Simple reference model/performance.
Checkpoint: Saved model state.
Class Imbalance: Unequal class distribution.
Data Leakage: Test information enters training.
Decision Boundary: Separates predicted classes.
Embedding Space: Vector space of learned meanings.
Feature: Input variable used by model.
Generalization: Performs well on unseen data.
Ground Truth: Correct reference answer/label.
Heuristic: Rule-of-thumb method.
Label: Correct output for supervised training.
Latent Space: Compressed hidden representation.
Model Architecture: Structure of model layers/components.
Objective Function: Function optimized during training.
Pretraining: Training on broad large data first.
Prompt: Input instructions sent to LLM.
Representation: Learned encoding of data.
Sampling: Choosing output tokens probabilistically.
Training Corpus: Dataset used for training.
Validation Set: Data used for tuning decisions.

Neural Network Terms

Activation Function: Adds nonlinearity to networks.
ReLU: Outputs max(0, x).
Sigmoid: Maps values to 0-1.
Tanh: Maps values to -1 to 1.
GELU: Smooth activation common in transformers.
Dead ReLU: ReLU stuck outputting zero.

Gradient: Direction/size of parameter update.
Vanishing Gradient: Gradients become too small.
Exploding Gradient: Gradients become too large.
Optimizer: Updates model weights.
AdamW: Adaptive optimizer with weight decay.
SGD: Basic gradient descent optimizer.
Weight Decay: Regularization penalizing large weights.
Batch Normalization: Normalizes layer inputs.
Layer Normalization: Normalizes features per sample.
Residual Block: Layer with skip connection.
Parameter Count: Number of learned weights.
Compute: Processing needed for training/inference.
FLOPs: Floating-point operations.
VRAM: GPU memory.

NLP and LLM Terms

NLP: AI for human language.
Token: Basic text unit processed by model.
Tokenizer: Converts text into tokens.
Vocabulary: Set of tokenizer tokens.
Subword Tokenization: Splits words into pieces.
BPE: Byte Pair Encoding tokenizer method.
WordPiece: Subword tokenizer used by BERT.
SentencePiece: Language-independent tokenizer.
Padding Token: Filler token for equal lengths.
Attention Mask: Marks real tokens vs padding.
Autoregressive Model: Predicts next token from previous tokens.
Masked Language Modeling: Predicts hidden tokens.
Next Sentence Prediction: Predicts sentence continuity.
Causal Language Modeling: Next-token prediction.
Bidirectional Attention: Looks left and right.
Unidirectional Attention: Looks only previous tokens.
Instruction Following: Model obeys user task instructions.
In-context Learning: Learns from prompt examples.
Context Length: Maximum tokens accepted.
Stop Sequence: Text that ends generation.
Logit: Raw score before probability.

Softmax: Converts scores to probabilities.
Decoding: Converts model probabilities to text.
Greedy Decoding: Always picks most likely token.
Beam Search: Keeps multiple candidate sequences.
Nucleus Sampling: Top-p sampling.

Generative AI Terms

Generative Model: Creates new data.
Discriminative Model: Predicts labels/classes.
Generator: GAN network that creates samples.
Discriminator: GAN network that judges samples.
Forward Diffusion: Adds noise to data.
Reverse Diffusion: Removes noise to generate.
Denoising: Removing noise from data.
Latent Diffusion: Diffusion in compressed space.
Multimodal Model: Handles multiple input types.
Image Captioning: Generates text description of image.
Text-to-Image: Generates image from prompt.
Image-to-Text: Generates text from image.
Speech-to-Text: Converts audio to text.
Text-to-Speech: Converts text to audio.
Synthetic Data: Artificially generated training data.
Hallucination: Plausible but incorrect generation.
Grounded Generation: Generation supported by evidence.

Evaluation Terms

Benchmark: Standard test for comparison.
Metric: Numeric performance measure.
Loss: Training error signal.
Train Loss: Error on training data.
Validation Loss: Error on validation data.
Test Accuracy: Final held-out accuracy.
False Positive: Incorrect positive prediction.
False Negative: Incorrect negative prediction.
True Positive: Correct positive prediction.
True Negative: Correct negative prediction.
ROC Curve: TPR/FPR curve across thresholds.

Confidence Score: Model certainty estimate.
Calibration Error: Confidence mismatch.
Inter-rater Agreement: Human judge consistency.
Regression Metric: Score for numeric prediction.
Classification Metric: Score for class prediction.
Retrieval Metric: Score for search quality.
Recall@k: Relevant item appears in top k.
MRR: Mean reciprocal rank.
NDCG: Ranking quality metric.

Data Science Terms

Grid Search: Tests all parameter combinations.
Randomized Search: Samples parameter combinations.
Hyperparameter Tuning: Finds best config.
Cross-validation Fold: One split in k-fold testing.
ARIMA: Time-series forecasting model.
Autocorrelation: Correlation with past values.
Partial Autocorrelation: Direct lag correlation.
Stationarity: Stable mean/variance over time.
Seasonality: Repeating time pattern.
Trend: Long-term direction.
Intercept: Baseline regression value.
Coefficient: Feature effect in regression.
Residual: Prediction error.
Outlier: Unusual data point.
Missing Value: Empty/unknown feature value.
Imputation: Fill missing values.
One-hot Encoding: Converts categories to binary columns.

NVIDIA and Production Terms

MIG: Splits one GPU into isolated instances.
Time-sharing: Multiple workloads share GPU time.
Model Signing: Cryptographic verification of model/container.
Vulnerability Scanning: Checks security weaknesses.
Version Pinning: Locks exact model/container version.
Helm: Kubernetes package manager.
Load Balancing: Distributes traffic across servers.

SLA: Required service reliability/performance.

Observability: Logs, metrics, traces for systems.

Telemetry: Collected runtime measurements.

Throughput Bottleneck: Component limiting system speed.

CPU-GPU Transfer: Data movement between CPU and GPU.

Zero-copy Interop: Avoids unnecessary data copying.

Python Tools and Libraries

Core Python ML Stack

Python: Main language for AI/ML workflows.

NumPy: Fast numerical arrays and math.

Pandas: CPU DataFrame data analysis.

Polars: Fast DataFrame processing library.

SciPy: Scientific computing and optimization.

scikit-learn: Classical ML and metrics toolkit.

Jupyter Notebook: Interactive coding and exploration.

Matplotlib: Basic Python plotting.

Seaborn: Statistical visualization on Matplotlib.

Plotly: Interactive charts and dashboards.

Statsmodels: Statistical models and time-series tools.

NLTK: Traditional NLP toolkit.

spaCy: Production NLP pipeline library.

Deep Learning Libraries

PyTorch: Flexible deep learning framework.

torch: PyTorch tensor and neural network package.

torch.nn: PyTorch neural network layers.

torch.optim: PyTorch optimizers.

torchvision: PyTorch image models/transforms.

torchaudio: PyTorch audio processing.

TensorFlow: Production deep learning framework.

Keras: High-level TensorFlow model API.

TensorBoard: Training visualization dashboard.

JAX: High-performance array/autodiff framework.

Autograd: Automatic gradient computation.

CUDA Tensor: Tensor stored on NVIDIA GPU.

Hugging Face Ecosystem

Transformers: Pretrained transformer models library.
AutoModel: Loads model architecture automatically.
AutoTokenizer: Loads matching tokenizer automatically.
Datasets: Dataset loading and preprocessing library.
Tokenizers: Fast tokenizer implementations.
Accelerate: Simplifies distributed/mixed-precision training.
PEFT: Parameter-efficient fine-tuning library.
TRL: RLHF/DPO training utilities.
Safetensors: Safe fast model weight format.
Model Hub: Repository for pretrained models/datasets.
Pipeline API: Quick task-based inference wrapper.

LLM App Development

LangChain: Builds LLM apps and chains.
LangGraph: Stateful graph workflows for agents.
LlamaIndex: Data connectors and RAG framework.
OpenAI Python SDK: Calls OpenAI-compatible APIs.
NVIDIA AI Endpoints: Hosted NVIDIA model APIs.
ChatNVIDIA: LangChain chat connector for NVIDIA.
NVIDIAEmbeddings: LangChain embedding connector.
FAISS Python: Local vector similarity search.
Milvus SDK: Connects Python to Milvus vector DB.
Pinecone SDK: Connects Python to Pinecone.
Weaviate Client: Connects Python to Weaviate.
Chroma: Lightweight local vector database.
SentenceTransformers: Embedding models for semantic search.
tiktoken: Token counting/tokenization utility.

NVIDIA Python and GPU Tools

RAPIDS: GPU data science Python ecosystem.
cuDF: GPU DataFrame library.
cudf.pandas: GPU acceleration for pandas code.
cuML: GPU machine learning algorithms.
cuGraph: GPU graph analytics.
Dask-cuDF: Distributed GPU DataFrames.
CuPy: NumPy-like arrays on GPU.

Numba: JIT compiler for Python/GPU kernels.
NVIDIA DALI: GPU data loading and preprocessing.
NVTabular: GPU tabular feature engineering.
NeMo Toolkit: Python framework for LLM customization.
NeMo Curator: Dataset curation and filtering.
NeMo Guardrails: Python safety/rail framework.
TensorRT Python API: Build optimized inference engines.
TensorRT-LLM: Optimized LLM inference library.
Triton Client: Python client for Triton inference.
NVIDIA NIM Client Pattern: Call NIM via REST/gRPC.
NCCL Bindings: Multi-GPU communication support.

Data Processing Libraries

PyArrow: Columnar data and Parquet support.
Parquet: Efficient columnar storage format.
JSONL: One JSON object per line.
CSV: Simple tabular text format.
BeautifulSoup: HTML parsing.
Requests: HTTP API calls.
aiohttp: Async HTTP client/server.
Regex: Pattern matching for text cleaning.
Unidecode: Text normalization utility.
pydantic: Data validation and schemas.

Serving and API Libraries

FastAPI: Modern Python REST API framework.
Flask: Lightweight Python web framework.
Uvicorn: ASGI server for FastAPI.
Gunicorn: Production Python web server.
gRPC Python: High-performance RPC services.
Streamlit: Quick Python data apps.
Gradio: Quick ML demos/interfaces.
ONNX Runtime: Runs ONNX models.
Triton Inference Server: Production model serving.
Docker SDK: Control Docker from Python.
Kubernetes Client: Manage Kubernetes from Python.

Testing and Quality Tools

pytest: Python testing framework.
unittest: Built-in Python test framework.
mypy: Static type checking.
ruff: Fast Python linter/formatter.
black: Opinionated Python formatter.
pre-commit: Runs checks before commits.
tox: Tests across environments.
coverage.py: Measures test coverage.
Great Expectations: Data quality validation.
Evidently: ML drift and monitoring reports.

Experiment Tracking and MLOps

MLflow: Tracks experiments and models.
Weights & Biases: Experiment tracking dashboard.
DVC: Data/model version control.
Hydra: Configuration management.
OmegaConf: Hierarchical config library.
Airflow: Workflow orchestration.
Prefect: Python workflow orchestration.
Ray: Distributed Python computing.
Ray Serve: Scalable model serving.
BentoML: Package and serve ML models.

Python Concepts for Certification

Virtual Environment: Isolates project dependencies.
pip: Python package installer.
conda: Environment and package manager.
requirements.txt: Lists pip dependencies.
pyproject.toml: Modern Python project config.
Asyncio: Python async concurrency.
Coroutine: Async function execution unit.
Type Hint: Optional Python type annotation.
Exception Handling: Handles runtime errors.
Serialization: Save/load data structures.
Pickle: Python object serialization.
Environment Variable: Runtime configuration value.
Logging Module: Built-in structured logging.

Exam Decision Tree

NVIDIA Tool Choice

Need deploy model fast: Choose NVIDIA NIM.
Need OpenAI-compatible endpoint: Choose NVIDIA NIM.
Need private/on-prem inference: Choose NVIDIA NIM.
Need optimize LLM inference: Choose TensorRT-LLM.
Need lower latency: Choose TensorRT-LLM or batching.
Need higher throughput: Choose TensorRT-LLM, Triton, batching.
Need serve many frameworks: Choose Triton.
Need model versioning/A-B tests: Choose Triton.
Need chain models server-side: Choose Triton ensemble.
Need train or fine-tune LLM: Choose NeMo.
Need RLHF/SFT/LoRA: Choose NeMo.
Need curate training data: Choose NeMo Curator.
Need GPU DataFrame processing: Choose RAPIDS/cuDF.
Need GPU ML algorithms: Choose cuML.
Need GPU graph analytics: Choose cuGraph.
Need safety/policy controls: Choose NeMo Guardrails.
Need block jailbreaks/injections: Choose Guardrails rails.
Need find NVIDIA containers/models: Choose NGC.
Need biology foundation models: Choose BioNeMo.
Need video AI streaming: Choose DeepStream.
Need GPU preprocessing: Choose DALI.

Model Method Choice

Need external/current knowledge: Use RAG.
Need domain behavior/style: Use fine-tuning.
Need cheap fine-tuning: Use PEFT/LoRA.
Need align to preferences: Use RLHF/DPO.
Need reduce model size: Use distillation/pruning/quantization.
Need faster decoding: Use KV cache/speculative decoding.
Need reduce memory: Use quantization/LoRA/checkpointing.
Need compare live versions: Use A/B testing.
Need unbiased final score: Use held-out test set.
Need robust estimate: Use cross-validation.

Common Exam Traps

RAG vs Fine-tuning: RAG adds knowledge; fine-tuning changes behavior/weights.
NIM vs NGC: NIM runs models; NGC stores models/containers.
NIM vs Triton: NIM is packaged service; Triton is serving infrastructure.
NeMo vs NIM: NeMo trains/customizes; NIM deploys.
TensorRT-LLM vs Triton: TensorRT optimizes; Triton serves.
Guardrails vs RLHF: Guardrails runtime controls; RLHF training alignment.
cuDF vs Pandas: cuDF GPU; pandas CPU.
cuML vs scikit-learn: cuML GPU ML; scikit-learn CPU ML.
Latency vs Throughput: Latency delay; throughput volume.
Precision vs Recall: Precision reduces false positives; recall reduces false negatives.
F1 vs Accuracy: F1 better for imbalance; accuracy can mislead.
BLEU vs ROUGE: BLEU translation; ROUGE summarization.
Perplexity vs Accuracy: Perplexity language uncertainty; accuracy exact correctness.
Encoder-only vs Decoder-only: Encoder understands; decoder generates.
BERT vs GPT: BERT bidirectional encoder; GPT autoregressive decoder.
Self-attention vs Cross-attention: Same sequence vs encoder-decoder link.
Top-k vs Top-p: Fixed count vs probability mass.
Temperature 0 vs High: Deterministic vs creative.
Quantization vs Pruning: Lower precision vs remove weights.
Data Parallelism vs Tensor Parallelism: Split data vs split model tensors.
Pipeline Parallelism vs Tensor Parallelism: Split layers vs split tensor operations.
Validation Set vs Test Set: Tune on validation; final score on test.
Data Bias vs Algorithmic Bias: Bad data vs biased model/design.
Prompt Injection vs Jailbreak: Hidden malicious instruction vs bypass attempt.
Groundedness vs Relevance: Supported by sources vs answers question.

Formula and Metric Mini Sheet

Classification

Accuracy: Correct predictions / all predictions.
Precision: TP / (TP + FP).
Recall: TP / (TP + FN).
F1: 2 * precision * recall / (precision + recall).
False Positive: Predicted positive, actually negative.
False Negative: Predicted negative, actually positive.
Confusion Matrix: Table of TP, FP, FN, TN.

AUC-ROC: Ranking quality across thresholds.

Regression

MAE: Average absolute error.

MSE: Average squared error.

RMSE: Square root of MSE.

R-squared: Proportion of variance explained.

NLP and LLM

Perplexity: Lower means better next-token prediction.

Cross-entropy: Penalizes wrong probability predictions.

BLEU: N-gram precision for translation.

ROUGE: N-gram recall for summarization.

METEOR: Translation metric with synonym/stem matching.

Groundedness: Claims supported by context.

Faithfulness: Output does not contradict source.

Toxicity: Harmful/offensive content score.

Retrieval and RAG

Cosine Similarity: Measures vector angle similarity.

Recall@k: Relevant item appears in top k.

Precision@k: Relevant fraction in top k.

MRR: Average reciprocal rank of first relevant result.

NDCG: Ranking quality with position weighting.

Production

Latency: Time for one request.

Throughput: Requests/tokens per second.

Error Rate: Failed requests / total requests.

GPU Utilization: Percent GPU busy.

Memory Footprint: RAM/VRAM required.

Tokens/sec: Generation speed.

Scenario Keywords

NVIDIA Tools

"OpenAI-compatible API": NIM.

"Pre-packaged optimized container": NIM.

"Run model locally/private cloud": NIM.

"Pull from nvcr.io": NGC/NIM.

"Catalog of models/containers": NGC.

"Helm chart": NGC/Kubernetes deployment.

"Fine-tune LLM": NeMo.

"SFT/RLHF/LoRA/P-Tuning": NeMo.

"Reward model/PPO": RLHF with NeMo.

"Data curation/filtering": NeMo Curator.

"Block unsafe topics": NeMo Guardrails.

"Colang": NeMo Guardrails.

"Input/output rails": NeMo Guardrails.

"Optimize LLM latency": TensorRT-LLM.

"Paged KV cache": TensorRT-LLM.

"INT4/INT8/FP8": TensorRT-LLM quantization.

"Dynamic batching": Triton.

"Model ensemble": Triton.

"Model versioning": Triton.

"Serve PyTorch/TensorFlow/ONNX together": Triton.

"GPU pandas": cuDF.

"GPU data science": RAPIDS.

"GPU clustering/PCA/nearest neighbors": cuML.

"GPU graph analytics": cuGraph.

Model and Data Methods

"Needs latest/private documents": RAG.

"Reduce hallucinations with sources": RAG.

"Vector database": Embeddings/RAG.

"Semantic search": Embeddings/vector DB.

"Small GPU memory for tuning": LoRA/QLoRA.

"Train only few parameters": PEFT.

"Human preferences": RLHF/DPO.

"Pairwise comparisons": Reward model/DPO.

"Reduce model size": Distillation/pruning/quantization.

"Faster inference": Quantization, TensorRT-LLM, batching.

"Multiple GPUs too large model": Tensor parallelism.

"Split layers across GPUs": Pipeline parallelism.

"Same model copy on each GPU": Data parallelism.

Evaluation and Safety

"Imbalanced classes": F1/precision/recall.
"False positives costly": Precision.
"False negatives costly": Recall.
"Translation quality": BLEU/METEOR.
"Summarization quality": ROUGE/human eval.
"Language model fluency": Perplexity.
"Open-ended answers": Human eval/LLM-as-judge.
"Production comparison": A/B testing.
"Final unbiased score": Hold-out test set.
"Distribution changes": Drift detection.
"Harmful prompts": Red teaming/adversarial testing.
"Sensitive user data": Privacy/PII controls.
"Demographic fairness": Bias/fairness evaluation.